

What is Slowly Changing Dimension (SCD)?

In **Data Warehousing**, a **Slowly Changing Dimension (SCD)** refers to a dimension table where **data changes slowly over time**.

Unlike transactional systems, where records are updated frequently, **dimension attributes change occasionally**.

Example:

Customer information

Customer_ID	Name	City
-------------	------	------

101	Rohith	Bangalore
-----	--------	-----------

If the customer moves to another city:

Customer_ID	Name	City
-------------	------	------

101	Rohith	Hyderabad
-----	--------	-----------

The question becomes:

How should we store this change in the data warehouse?

Different methods to handle this are called **SCD Types**.

Why SCD is Important

SCD helps to:

- Track historical data changes
- Maintain accurate reporting
- Analyze trends over time
- Preserve previous values of dimension attributes

Example analysis:

Customer location changes over time

Employee department history

Product price changes

Types of Slowly Changing Dimensions

Most commonly used types:

Type 0
Type 1
Type 2
Type 3
Type 4
Type 6 (Hybrid)

In real projects, **Type 1 and Type 2** are used most frequently.

SCD Type 0 – Fixed Dimension

In **Type 0**, the dimension value **never changes**.

If new data arrives, it is **ignored**.

Example:

Date of Birth

Original Registration Date

Example table:

Customer_ID	Name	DOB
-------------	------	-----

101	Rohith	01-01-1998
-----	--------	------------

Even if a new DOB arrives, the **original value remains unchanged**.

SCD Type 1 – Overwrite

In **Type 1**, the existing data is **updated directly**.

Old data is **lost**.

Example:

Customer moves from Bangalore to Hyderabad.

Before update

Customer_ID	City
-------------	------

101	Bangalore
-----	-----------

After update

Customer_ID	City
-------------	------

101	Hyderabad
-----	-----------

Implementation:

```
UPDATE customer_dimension  
SET city = 'Hyderabad'  
WHERE customer_id = 101;
```

Characteristics:

- No history maintained
- Simple implementation
- Used when history is not important

Example use cases:

Email address
Phone number corrections

SCD Type 2 – Maintain Full History

Type 2 **preserves historical data** by creating **new rows** for changes.

Each change generates a **new record**.

Extra columns are used to track history:

- Start_Date
- End_Date
- Is_Current_Flag
- Version

Example:

Customer changes city.

Before change

Cust_ID	City	Start_Date	End_Date	Current
101	Bangalore	2023	NULL	Y

After change

Cust_ID	City	Start_Date	End_Date	Current
101	Bangalore	2023	2024	N
101	Hyderabad	2024	NULL	Y

Implementation steps:

1. Detect change
2. Update existing record End_Date
3. Insert new record

Example SQL logic:

```
UPDATE customer_dim  
SET end_date = CURRENT_DATE,  
current_flag = 'N'  
WHERE customer_id = 101  
AND current_flag = 'Y';
```

Then insert new row.

```
INSERT INTO customer_dim  
VALUES (101,'Hyderabad',CURRENT_DATE,NULL,'Y');
```

Characteristics:

- Full historical tracking
 - Most widely used in Data Warehouses
-

SCD Type 3 – Limited History

Type 3 stores **limited history using extra columns**.

Example columns:

- Current_City
- Previous_City

Example table:

Customer_ID	Current_City	Previous_City
-------------	--------------	---------------

101	Hyderabad	Bangalore
-----	-----------	-----------

When change occurs:

Old value moves to **previous column**.

Limitation:

Only **one historical value stored**.

SCD Type 4 – History Table

Type 4 separates:

- Current data
- Historical data

Two tables exist.

Current Dimension Table

Cust_ID City

101 Hyderabad

History Table

Cust_ID City Change_Date

101 Bangalore 2023

Used when **history becomes very large**.

SCD Type 6 – Hybrid (Type 1 + Type 2 + Type 3)

Type 6 combines multiple SCD methods.

It includes:

- New row insertion (Type 2)
- Column overwrite (Type 1)
- Previous value column (Type 3)

Example table

Cust_ID City Prev_City Start_Date End_Date Current

101 Hyderabad Bangalore 2024 NULL Y

This provides **complete history + quick access to previous value**.

Real-Time Example – Customer Dimension

In a **Retail Data Warehouse**.

Customer moves cities frequently.

Business wants:

- Current customer location
- Historical location changes

Best solution:

SCD Type 2

Because it preserves full history.

SCD Implementation in ETL / ELT

Typical pipeline:

Source System

↓

Staging Table

↓

Compare with Dimension Table

↓

Detect Changes

↓

Apply SCD logic

Steps:

1. Load new data to staging table
2. Compare staging vs dimension table
3. Identify changed rows
4. Apply Type1 / Type2 logic
5. Load into dimension table

SCD Implementation in Snowflake

Snowflake commonly uses:

MERGE command.

Example:

```
MERGE INTO customer_dim t
USING staging_customer s
ON t.customer_id = s.customer_id
AND t.current_flag = 'Y'
```

```
WHEN MATCHED AND t.city <> s.city THEN
UPDATE SET
t.end_date = CURRENT_DATE,
t.current_flag = 'N'
```

```
WHEN NOT MATCHED THEN
INSERT
(customer_id, city, start_date, end_date, current_flag)
VALUES
(s.customer_id, s.city, CURRENT_DATE, NULL, 'Y');
```

This is the **most common SCD implementation in Snowflake pipelines**.

SCD Comparison Table

Type	History	Method
Type 0	No change	Ignore update
Type 1	No history	Overwrite
Type 2	Full history	New row
Type 3	Limited history	Extra column
Type 4	Separate history table	Archive
Type 6	Hybrid	Combine methods

Most Used SCD Types in Real Projects

Industry usage:

Type 1 → ~40%

Type 2 → ~50%

Type 3 → Rare

Type 4 → Special cases

Type 6 → Advanced models

Typical Interview Questions

What is SCD?

Slowly Changing Dimension is a method used in data warehouses to **manage changes in dimension attributes over time**.

Which SCD type preserves full history?

Type 2

Which SCD type overwrites old data?

Type 1

Which SCD type stores previous value in another column?

Type 3

Which SCD type is most used in Data Warehousing?

Type 2

Quick Summary

Slowly Changing Dimensions help **track and manage historical changes in dimension tables**.

Different SCD types define **how data changes are stored**, with **Type 1 and Type 2** being the most **widely used methods in real data warehouse projects**.

ROHITH KUMAR